READ DATA

```
import os
import random
from google.colab import drive

drive.mount('/content/drive')
train_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/train/PNEUMONIA'
test_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/test/PNEUMONIA'
train_normal_dir = '/content/drive/MyDrive/TA/Dataset/train/NORMAL'
test_normal_dir = '/content/drive/MyDrive/TA/Dataset/test/NORMAL'
```

```
Mounted at /content/drive
```

```
def get_all_files(dirs, allowed_extensions=['.jpg', '.jpeg', '.png']):
    all_files = []
    for dir_path in dirs:
        files = [os.path.join(dir_path, f) for f in os.listdir(dir_path)
                 if not f.startswith('.') and os.path.splitext(f)[1].lower() in allowed_extensions]
        all_files.extend(files)
    return all_files

def random_select_files(file_list, subset_size):
    if subset_size > len(file_list):
        raise ValueError("Jumlah file yang diminta lebih besar daripada jumlah file yang tersedia.")
    return random.sample(file_list, subset_size)

def count_files(directory):
  return len(os.listdir(directory))
```

```
all_pneumonia_files = get_all_files([train_pneumonia_dir, test_pneumonia_dir])
all_normal_files = get_all_files([train_normal_dir, test_normal_dir])

print(f"train pneumon : {count_files(train_pneumonia_dir)}")
print(f"test pneumon : {count_files(test_pneumonia_dir)}")
print(f"train norm : {count_files(train_normal_dir)}")
print(f"test norm : {count_files(test_normal_dir)}")

print(f"pneumon : {len(all_pneumonia_files)}")
```

```
print(f"norm : {len(all_normal_files)}")
```

```
train pneumon : 3875
test pneumon : 390
train norm : 1341
test norm : 234
pneumon : 4265
norm : 1575
```

BALANCING

```
train_pneumonia_count = 3200
test_pneumonia_count = 1065
train_normal_count = 3200
test_normal_count = 1065

test_pneumonia_samples = random.sample(all_pneumonia_files, test_pneumonia_count)
test_normal_samples = random.sample(all_normal_files, test_normal_count)

pneumonia_train_pool = [f for f in all_pneumonia_files if f not in test_pneumonia_samples]
normal_train_pool = [f for f in all_normal_files if f not in test_normal_samples]

train_pneumonia_samples = random.choices(pneumonia_train_pool, k=train_pneumonia_count)
train_normal_samples = random.choices(normal_train_pool, k=train_normal_count)
```

```
print(f"train_pneumonia_samples : {len(train_pneumonia_samples)}")
print(f"test_pneumonia_samples : {len(test_pneumonia_samples)}")
print(f"train_normal_samples : {len(train_normal_samples)}")
print(f"test_normal_samples : {len(test_normal_samples)}")
```

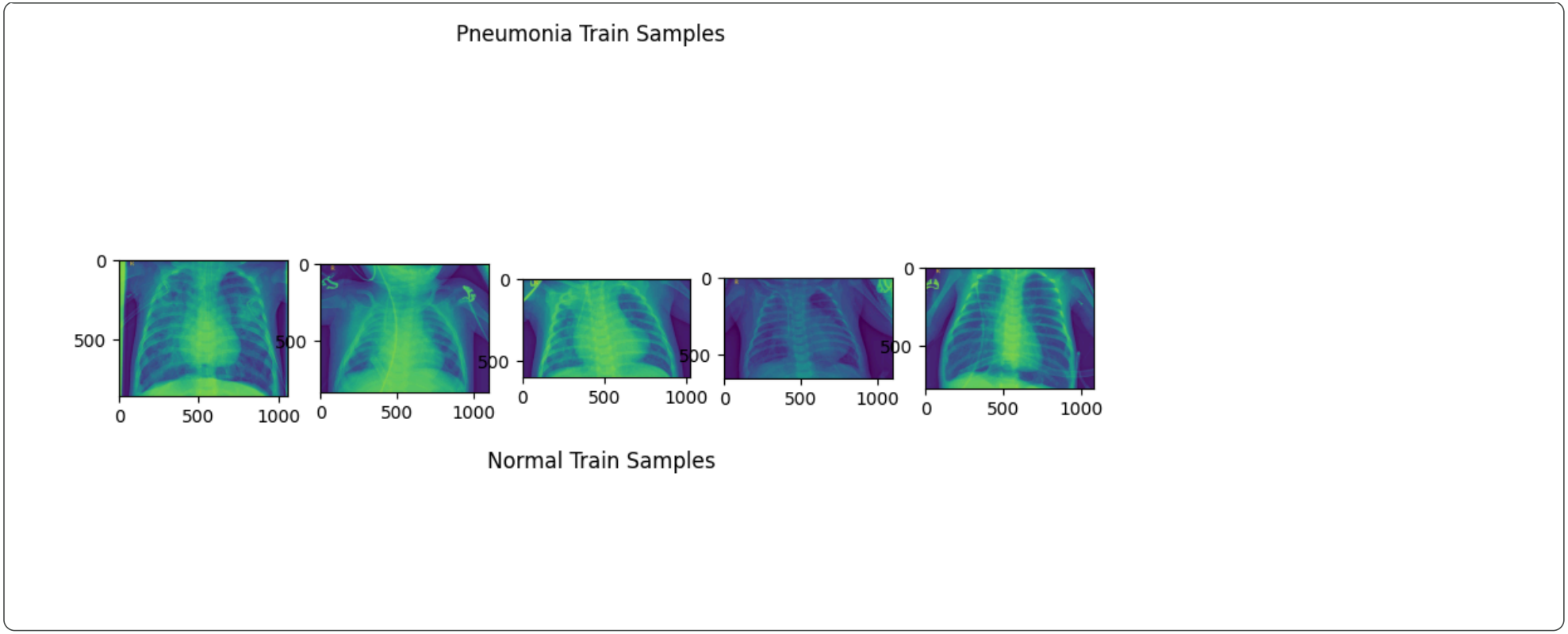
```
train_pneumonia_samples : 3200
test_pneumonia_samples : 1065
train_normal_samples : 3200
test_normal_samples : 1065
```

```
from PIL import Image
import matplotlib.pyplot as plt

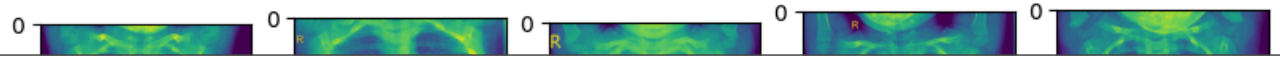
def show_sample_images(sample_files,ammount, title):
    plt.figure(figsize=(10, 5))
    plt.suptitle(title)
    for i in range(ammount):
        plt.subplot(1, ammount, i+1)
```

```
            img = Image.open(sample_files[i])
            plt.imshow(img)
        plt.show()

    show_sample_images(train_pneumonia_samples, 5, 'Pneumonia Train Samples')
    show_sample_images(train_normal_samples, 5, 'Normal Train Samples')
    show_sample_images(test_pneumonia_samples, 5, 'Pneumonia Test Samples')
    show_sample_images(test_normal_samples, 5, 'Normal Test Samples')
```



RESIZE



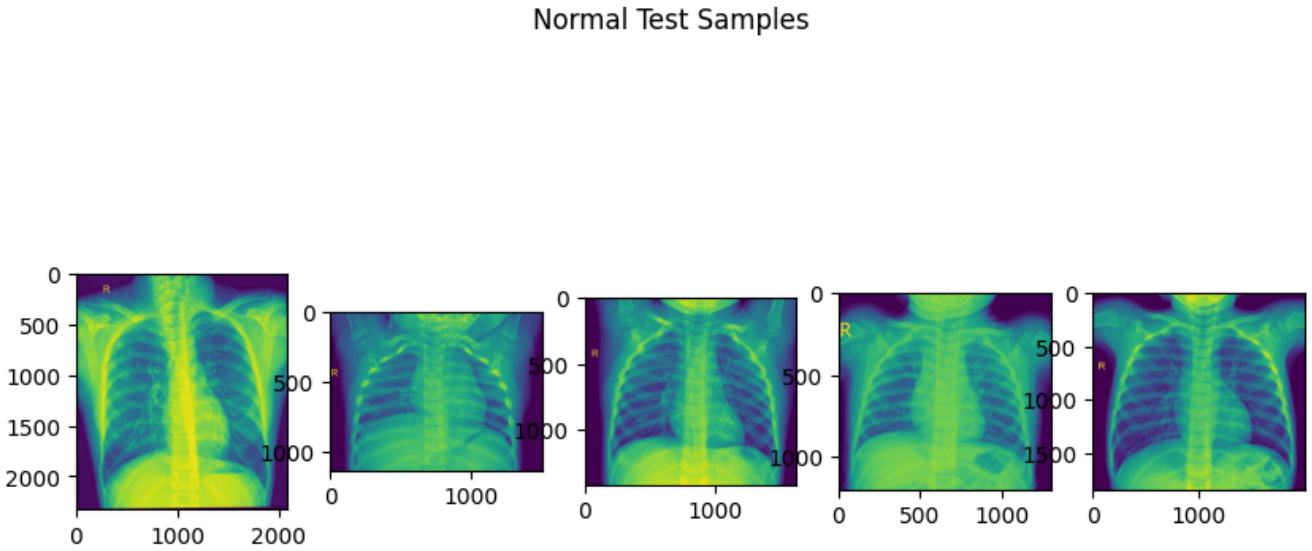
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array

def load_images(file_paths, target_size=(224, 224)):
    images = []
    for i, file_path in enumerate(file_paths):
        img = load_img(file_path, target_size=target_size)
        img_array = img_to_array(img)
        img_array = img_array - 30
        img_array = img_array / 255.0
        images.append(img_array)
    return images

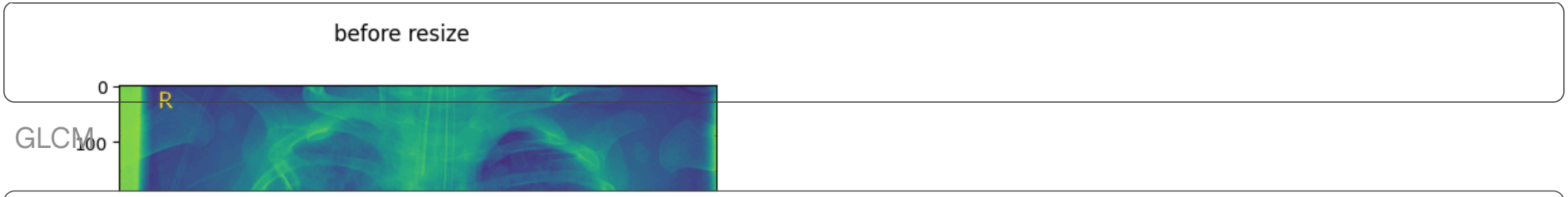
train_pneumonia_images = load_images(train_pneumonia_samples)
test_pneumonia_images = load_images(test_pneumonia_samples)
```

```
train_normal_images = load_images(train_normal_samples)
test_normal_images = load_images(test_normal_samples)
```

```
show_sample_images(train_pneumonia_samples, 1, 'before resize')
plt.imshow(train_pneumonia_images[0])
plt.title('After Resizing')
```







GLCM

```
import numpy as np
import cv2
from skimage.feature import graycomatrix, graycoprops

def calculate_glcm(image):
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    distances = [1, 2, 3]
    angles = [0, np.pi/4, np.pi/2, 3*np.pi/4]

    glcm = graycomatrix(gray_image,
                        distances=distances,
                        angles=angles,
                        levels=256,
                        symmetric=True,
                        normed=True)

    properties = ['contrast', 'dissimilarity', 'homogeneity', 'energy', 'correlation', 'ASM']
    features = {}

    for prop in properties:
        values = graycoprops(glcm, prop).flatten()
        features[f'{prop}_mean'] = np.mean(values)
        features[f'{prop}_std'] = np.std(values)
        features[f'{prop}_var'] = np.var(values)

    glcm_entropy = -np.sum(glcm * np.log2(glcm + 1e-10))
    features['entropy'] = glcm_entropy

    return features
```



```
import cv2

def extract_glcm_features(image_list):
    glcm_features_list = []
    for image in image_list:
        image_uint8 = (image * 255).astype(np.uint8)
        features = calculate_glcm(image_uint8)
```

```
            glcm_features_list.append(features)
        return glcm_features_list

    train_pneumonia_glcm = extract_glcm_features(train_pneumonia_images)
    test_pneumonia_glcm = extract_glcm_features(test_pneumonia_images)
    train_normal_glcm = extract_glcm_features(train_normal_images)
    test_normal_glcm = extract_glcm_features(test_normal_images)

    print("GLCM features for the first pneumonia training image:")
    train_pneumonia_glcm[0]
```

```
GLCM features for the first pneumonia training image:
{'contrast_mean': np.float64(1215.7637282097342),
 'contrast_std': np.float64(604.2551625381589),
 'contrast_var': np.float64(365124.30145401676),
 'dissimilarity_mean': np.float64(13.446299555270656),
 'dissimilarity_std': np.float64(4.462842562958955),
 'dissimilarity_var': np.float64(19.916963741758057),
 'homogeneity_mean': np.float64(0.16560993036757773),
 'homogeneity_std': np.float64(0.04336721745535526),
 'homogeneity_var': np.float64(0.0018807155498200701),
 'energy_mean': np.float64(0.014852845892011206),
 'energy_std': np.float64(0.001976914214063014),
 'energy_var': np.float64(3.908189809764385e-06),
 'correlation_mean': np.float64(0.8109860402664245),
 'correlation_std': np.float64(0.09408777855407914),
 'correlation_var': np.float64(0.008852510073241435),
 'ASM_mean': np.float64(0.00022451522090159858),
 'ASM_std': np.float64(6.113779159230574e-05),
 'ASM_var': np.float64(3.7378295607842105e-09),
 'entropy': np.float64(153.51413650128237)}
```

```
#import pandas as pd
#from google.colab import files

#train_pneumonia_df = pd.DataFrame(train_pneumonia_glcm)
#train_pneumonia_df['label'] = 'PNEUMONIA'

#test_pneumonia_df = pd.DataFrame(test_pneumonia_glcm)
#test_pneumonia_df['label'] = 'PNEUMONIA'

#train_normal_df = pd.DataFrame(train_normal_glcm)
#train_normal_df['label'] = 'NORMAL'

#test_normal_df = pd.DataFrame(test_normal_glcm)
#test_normal_df['label'] = 'NORMAL'
```

```
#all_glcm_df = pd.concat([train_pneumonia_df, test_pneumonia_df, train_normal_df, test_normal_df], ignore_index=True)

#excel_filename = 'glcm_features.xlsx'
#all_glcm_df.to_excel(excel_filename, index=False)

#files.download(excel_filename)

#print(f"GLCM features saved to '{excel_filename}' and downloaded.")
```

CNN

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.regularizers import l2

# GLCM - DF
train_pneumonia_glcm_df = pd.DataFrame(train_pneumonia_glcm)
test_pneumonia_glcm_df = pd.DataFrame(test_pneumonia_glcm)
train_normal_glcm_df = pd.DataFrame(train_normal_glcm)
test_normal_glcm_df = pd.DataFrame(test_normal_glcm)

train_pneumonia_glcm_df['label'] = 1  # 1 for pneumonia
test_pneumonia_glcm_df['label'] = 1
train_normal_glcm_df['label'] = 0  # 0 for normal
test_normal_glcm_df['label'] = 0

train_glcm_df = pd.concat([train_pneumonia_glcm_df, train_normal_glcm_df], ignore_index=True)
test_glcm_df = pd.concat([test_pneumonia_glcm_df, test_normal_glcm_df], ignore_index=True)


# Separate features
X_train = train_glcm_df.drop('label', axis=1)
y_train = train_glcm_df['label']
X_test = test_glcm_df.drop('label', axis=1)
y_test = test_glcm_df['label']

# Normal
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Model
model = Sequential([
    Dense(128, activation='relu', kernel_regularizer=l2(0.0001), input_shape=(X_train.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(64, activation='relu', kernel_regularizer=l2(0.0001)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(32, activation='relu', kernel_regularizer=l2(0.0001)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])

optimizer = Adam(learning_rate=0.001)
class_weights = {0: 1.5, 1: 0.8}

callbacks = [
    EarlyStopping(monitor='val_loss', patience=50, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=25, min_lr=1e-6)
]

model.compile(optimizer=optimizer,
              loss='binary_crossentropy',
              metrics=['accuracy', tf.keras.metrics.AUC()])

model.summary()

history = model.fit(
    X_train, y_train,
    #epochs=50,
    epochs=100,
    batch_size=32,
    validation_split=0.3,
    #callbacks=None,
    callbacks=callbacks,
    class_weight=class_weights,
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_5"
```

Layer (type)	Output Shape	Param #
dense_14 (Dense)	(None, 128)	2,560
batch_normalization_9 (BatchNormalization)	(None, 128)	512
dropout_9 (Dropout)	(None, 128)	0
dense_15 (Dense)	(None, 64)	8,256
batch_normalization_10 (BatchNormalization)	(None, 64)	256
dropout_10 (Dropout)	(None, 64)	0
dense_16 (Dense)	(None, 32)	2,080
batch_normalization_11 (BatchNormalization)	(None, 32)	128
dropout_11 (Dropout)	(None, 32)	0
dense_17 (Dense)	(None, 1)	33

```
 Total params: 13,825 (54.00 KB)
 Trainable params: 13,377 (52.25 KB)
 Non-trainable params: 448 (1.75 KB)
Epoch 1/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 8s 19ms/step - accuracy: 0.7589 - auc_5: 0.8669 - loss: 0.5025 - val_accuracy: 0.8870 - val_auc_5: 0.0000e+00 - val_loss: 0.45
Epoch 2/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8353 - auc_5: 0.9060 - loss: 0.4053 - val_accuracy: 0.8984 - val_auc_5: 0.0000e+00 - val_loss: 0.337
Epoch 3/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8598 - auc_5: 0.9193 - loss: 0.3721 - val_accuracy: 0.9047 - val_auc_5: 0.0000e+00 - val_loss: 0.328
Epoch 4/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8674 - auc_5: 0.9278 - loss: 0.3500 - val_accuracy: 0.9099 - val_auc_5: 0.0000e+00 - val_loss: 0.306
Epoch 5/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8701 - auc_5: 0.9333 - loss: 0.3387 - val_accuracy: 0.9115 - val_auc_5: 0.0000e+00 - val_loss: 0.310
Epoch 6/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8754 - auc_5: 0.9345 - loss: 0.3317 - val_accuracy: 0.9161 - val_auc_5: 0.0000e+00 - val_loss: 0.270
Epoch 7/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8746 - auc_5: 0.9361 - loss: 0.3291 - val_accuracy: 0.9120 - val_auc_5: 0.0000e+00 - val_loss: 0.306
Epoch 8/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8728 - auc_5: 0.9388 - loss: 0.3242 - val_accuracy: 0.9073 - val_auc_5: 0.0000e+00 - val_loss: 0.298
Epoch 9/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8777 - auc_5: 0.9379 - loss: 0.3244 - val_accuracy: 0.9094 - val_auc_5: 0.0000e+00 - val_loss: 0.278
Epoch 10/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8768 - auc_5: 0.9419 - loss: 0.3168 - val_accuracy: 0.9245 - val_auc_5: 0.0000e+00 - val_loss: 0.264
Epoch 11/100
```

```
Epoch 11/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8826 - auc_5: 0.9441 - loss: 0.3100 - val_accuracy: 0.9271 - val_auc_5: 0.0000e+00 - val_loss: 0.276
Epoch 12/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8873 - auc_5: 0.9462 - loss: 0.3014 - val_accuracy: 0.9250 - val_auc_5: 0.0000e+00 - val_loss: 0.250
Epoch 13/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8859 - auc_5: 0.9487 - loss: 0.2992 - val_accuracy: 0.9260 - val_auc_5: 0.0000e+00 - val_loss: 0.256
Epoch 14/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8855 - auc_5: 0.9473 - loss: 0.3017 - val_accuracy: 0.9115 - val_auc_5: 0.0000e+00 - val_loss: 0.291
Epoch 15/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8884 - auc_5: 0.9486 - loss: 0.2978 - val_accuracy: 0.9224 - val_auc_5: 0.0000e+00 - val_loss: 0.269
Epoch 16/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8882 - auc_5: 0.9478 - loss: 0.2978 - val_accuracy: 0.9297 - val_auc_5: 0.0000e+00 - val_loss: 0.250
Epoch 17/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8891 - auc_5: 0.9505 - loss: 0.2925 - val_accuracy: 0.9214 - val_auc_5: 0.0000e+00 - val_loss: 0.268
Epoch 18/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8873 - auc_5: 0.9501 - loss: 0.2931 - val_accuracy: 0.9260 - val_auc_5: 0.0000e+00 - val_loss: 0.265
Epoch 19/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8922 - auc_5: 0.9521 - loss: 0.2866 - val_accuracy: 0.9328 - val_auc_5: 0.0000e+00 - val_loss: 0.251
Epoch 20/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8969 - auc_5: 0.9531 - loss: 0.2861 - val_accuracy: 0.9286 - val_auc_5: 0.0000e+00 - val_loss: 0.246
Epoch 21/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8931 - auc_5: 0.9515 - loss: 0.2891 - val_accuracy: 0.9240 - val_auc_5: 0.0000e+00 - val_loss: 0.273
Epoch 22/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8942 - auc_5: 0.9558 - loss: 0.2780 - val_accuracy: 0.9349 - val_auc_5: 0.0000e+00 - val_loss: 0.245
Epoch 23/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8951 - auc_5: 0.9576 - loss: 0.2717 - val_accuracy: 0.9281 - val_auc_5: 0.0000e+00 - val_loss: 0.260
Epoch 24/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8933 - auc_5: 0.9544 - loss: 0.2803 - val_accuracy: 0.9245 - val_auc_5: 0.0000e+00 - val_loss: 0.283
Epoch 25/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8951 - auc_5: 0.9548 - loss: 0.2827 - val_accuracy: 0.9286 - val_auc_5: 0.0000e+00 - val_loss: 0.254
Epoch 26/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8984 - auc_5: 0.9547 - loss: 0.2805 - val_accuracy: 0.9021 - val_auc_5: 0.0000e+00 - val_loss: 0.307
Epoch 27/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8958 - auc_5: 0.9562 - loss: 0.2778 - val_accuracy: 0.9333 - val_auc_5: 0.0000e+00 - val_loss: 0.244
Epoch 28/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8926 - auc_5: 0.9543 - loss: 0.2811 - val_accuracy: 0.9224 - val_auc_5: 0.0000e+00 - val_loss: 0.238
Epoch 29/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9002 - auc_5: 0.9581 - loss: 0.2716 - val_accuracy: 0.9464 - val_auc_5: 0.0000e+00 - val_loss: 0.239
Epoch 30/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8993 - auc_5: 0.9581 - loss: 0.2691 - val_accuracy: 0.9385 - val_auc_5: 0.0000e+00 - val_loss: 0.233
Epoch 31/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9033 - auc_5: 0.9587 - loss: 0.2671 - val_accuracy: 0.9323 - val_auc_5: 0.0000e+00 - val_loss: 0.256
Epoch 32/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8993 - auc_5: 0.9561 - loss: 0.2741 - val_accuracy: 0.9266 - val_auc_5: 0.0000e+00 - val_loss: 0.232
Epoch 33/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9011 - auc_5: 0.9597 - loss: 0.2634 - val_accuracy: 0.9401 - val_auc_5: 0.0000e+00 - val_loss: 0.218
Epoch 34/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8958 - auc_5: 0.9595 - loss: 0.2663 - val_accuracy: 0.9203 - val_auc_5: 0.0000e+00 - val_loss: 0.259
Epoch 35/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9018 - auc_5: 0.9610 - loss: 0.2577 - val_accuracy: 0.9203 - val_auc_5: 0.0000e+00 - val_loss: 0.240
Epoch 36/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9056 - auc_5: 0.9626 - loss: 0.2547 - val_accuracy: 0.9323 - val_auc_5: 0.0000e+00 - val_loss: 0.248
Epoch 37/100
```

```
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8991 - auc_5: 0.9614 - loss: 0.2595 - val_accuracy: 0.9219 - val_auc_5: 0.0000e+00 - val_loss: 0.244
Epoch 38/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9004 - auc_5: 0.9618 - loss: 0.2610 - val_accuracy: 0.9495 - val_auc_5: 0.0000e+00 - val_loss: 0.210
Epoch 39/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9016 - auc_5: 0.9607 - loss: 0.2608 - val_accuracy: 0.9344 - val_auc_5: 0.0000e+00 - val_loss: 0.224
Epoch 40/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9025 - auc_5: 0.9610 - loss: 0.2596 - val_accuracy: 0.9349 - val_auc_5: 0.0000e+00 - val_loss: 0.239
Epoch 41/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9002 - auc_5: 0.9617 - loss: 0.2576 - val_accuracy: 0.9120 - val_auc_5: 0.0000e+00 - val_loss: 0.258
Epoch 42/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9009 - auc_5: 0.9596 - loss: 0.2641 - val_accuracy: 0.9370 - val_auc_5: 0.0000e+00 - val_loss: 0.230
Epoch 43/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9049 - auc_5: 0.9630 - loss: 0.2546 - val_accuracy: 0.9406 - val_auc_5: 0.0000e+00 - val_loss: 0.197
Epoch 44/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9040 - auc_5: 0.9629 - loss: 0.2544 - val_accuracy: 0.9417 - val_auc_5: 0.0000e+00 - val_loss: 0.201
Epoch 45/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9114 - auc_5: 0.9642 - loss: 0.2502 - val_accuracy: 0.9406 - val_auc_5: 0.0000e+00 - val_loss: 0.225
Epoch 46/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9049 - auc_5: 0.9650 - loss: 0.2483 - val_accuracy: 0.9385 - val_auc_5: 0.0000e+00 - val_loss: 0.223
Epoch 47/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9007 - auc_5: 0.9623 - loss: 0.2581 - val_accuracy: 0.9292 - val_auc_5: 0.0000e+00 - val_loss: 0.242
Epoch 48/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9062 - auc_5: 0.9643 - loss: 0.2485 - val_accuracy: 0.9396 - val_auc_5: 0.0000e+00 - val_loss: 0.240
Epoch 49/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9045 - auc_5: 0.9626 - loss: 0.2557 - val_accuracy: 0.9625 - val_auc_5: 0.0000e+00 - val_loss: 0.203
Epoch 50/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9062 - auc_5: 0.9657 - loss: 0.2461 - val_accuracy: 0.9396 - val_auc_5: 0.0000e+00 - val_loss: 0.215
Epoch 51/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8987 - auc_5: 0.9610 - loss: 0.2601 - val_accuracy: 0.9448 - val_auc_5: 0.0000e+00 - val_loss: 0.227
Epoch 52/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9056 - auc_5: 0.9663 - loss: 0.2459 - val_accuracy: 0.9448 - val_auc_5: 0.0000e+00 - val_loss: 0.214
Epoch 53/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9103 - auc_5: 0.9658 - loss: 0.2442 - val_accuracy: 0.9380 - val_auc_5: 0.0000e+00 - val_loss: 0.202
Epoch 54/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9112 - auc_5: 0.9675 - loss: 0.2379 - val_accuracy: 0.9271 - val_auc_5: 0.0000e+00 - val_loss: 0.230
Epoch 55/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9056 - auc_5: 0.9647 - loss: 0.2482 - val_accuracy: 0.9208 - val_auc_5: 0.0000e+00 - val_loss: 0.224
Epoch 56/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9129 - auc_5: 0.9683 - loss: 0.2341 - val_accuracy: 0.9214 - val_auc_5: 0.0000e+00 - val_loss: 0.232
Epoch 57/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9132 - auc_5: 0.9685 - loss: 0.2338 - val_accuracy: 0.9411 - val_auc_5: 0.0000e+00 - val_loss: 0.200
Epoch 58/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9161 - auc_5: 0.9669 - loss: 0.2407 - val_accuracy: 0.9354 - val_auc_5: 0.0000e+00 - val_loss: 0.199
Epoch 59/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9067 - auc_5: 0.9653 - loss: 0.2446 - val_accuracy: 0.9422 - val_auc_5: 0.0000e+00 - val_loss: 0.211
Epoch 60/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9116 - auc_5: 0.9689 - loss: 0.2328 - val_accuracy: 0.9203 - val_auc_5: 0.0000e+00 - val_loss: 0.258
Epoch 61/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9078 - auc_5: 0.9637 - loss: 0.2488 - val_accuracy: 0.9609 - val_auc_5: 0.0000e+00 - val_loss: 0.187
Epoch 62/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9121 - auc_5: 0.9683 - loss: 0.2366 - val_accuracy: 0.9292 - val_auc_5: 0.0000e+00 - val_loss: 0.211
Epoch 63/100
```

```
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9123 - auc_5: 0.9693 - loss: 0.2330 - val_accuracy: 0.9547 - val_auc_5: 0.0000e+00 - val_loss: 0.205
Epoch 64/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9118 - auc_5: 0.9665 - loss: 0.2426 - val_accuracy: 0.9521 - val_auc_5: 0.0000e+00 - val_loss: 0.200
Epoch 65/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9170 - auc_5: 0.9699 - loss: 0.2291 - val_accuracy: 0.9438 - val_auc_5: 0.0000e+00 - val_loss: 0.197
Epoch 66/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9179 - auc_5: 0.9689 - loss: 0.2302 - val_accuracy: 0.9625 - val_auc_5: 0.0000e+00 - val_loss: 0.177
Epoch 67/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9100 - auc_5: 0.9645 - loss: 0.2513 - val_accuracy: 0.9427 - val_auc_5: 0.0000e+00 - val_loss: 0.202
Epoch 68/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9167 - auc_5: 0.9708 - loss: 0.2271 - val_accuracy: 0.9573 - val_auc_5: 0.0000e+00 - val_loss: 0.194
Epoch 69/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9147 - auc_5: 0.9702 - loss: 0.2277 - val_accuracy: 0.9432 - val_auc_5: 0.0000e+00 - val_loss: 0.210
Epoch 70/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9163 - auc_5: 0.9701 - loss: 0.2306 - val_accuracy: 0.9505 - val_auc_5: 0.0000e+00 - val_loss: 0.191
Epoch 71/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9165 - auc_5: 0.9703 - loss: 0.2282 - val_accuracy: 0.9281 - val_auc_5: 0.0000e+00 - val_loss: 0.215
Epoch 72/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9150 - auc_5: 0.9692 - loss: 0.2314 - val_accuracy: 0.9255 - val_auc_5: 0.0000e+00 - val_loss: 0.219
Epoch 73/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9089 - auc_5: 0.9676 - loss: 0.2375 - val_accuracy: 0.9552 - val_auc_5: 0.0000e+00 - val_loss: 0.182
Epoch 74/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9112 - auc_5: 0.9680 - loss: 0.2344 - val_accuracy: 0.9531 - val_auc_5: 0.0000e+00 - val_loss: 0.180
Epoch 75/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9118 - auc_5: 0.9676 - loss: 0.2375 - val_accuracy: 0.9307 - val_auc_5: 0.0000e+00 - val_loss: 0.215
Epoch 76/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9118 - auc_5: 0.9702 - loss: 0.2297 - val_accuracy: 0.9438 - val_auc_5: 0.0000e+00 - val_loss: 0.219
Epoch 77/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9152 - auc_5: 0.9709 - loss: 0.2296 - val_accuracy: 0.9495 - val_auc_5: 0.0000e+00 - val_loss: 0.188
Epoch 78/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9147 - auc_5: 0.9705 - loss: 0.2299 - val_accuracy: 0.9568 - val_auc_5: 0.0000e+00 - val_loss: 0.206
Epoch 79/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9210 - auc_5: 0.9727 - loss: 0.2187 - val_accuracy: 0.9583 - val_auc_5: 0.0000e+00 - val_loss: 0.180
Epoch 80/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9179 - auc_5: 0.9713 - loss: 0.2251 - val_accuracy: 0.9500 - val_auc_5: 0.0000e+00 - val_loss: 0.180
Epoch 81/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9163 - auc_5: 0.9719 - loss: 0.2232 - val_accuracy: 0.9661 - val_auc_5: 0.0000e+00 - val_loss: 0.169
Epoch 82/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9132 - auc_5: 0.9670 - loss: 0.2401 - val_accuracy: 0.9656 - val_auc_5: 0.0000e+00 - val_loss: 0.197
Epoch 83/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9192 - auc_5: 0.9728 - loss: 0.2210 - val_accuracy: 0.9620 - val_auc_5: 0.0000e+00 - val_loss: 0.172
Epoch 84/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9234 - auc_5: 0.9732 - loss: 0.2179 - val_accuracy: 0.9641 - val_auc_5: 0.0000e+00 - val_loss: 0.174
Epoch 85/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9187 - auc_5: 0.9723 - loss: 0.2208 - val_accuracy: 0.9516 - val_auc_5: 0.0000e+00 - val_loss: 0.196
Epoch 86/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9163 - auc_5: 0.9723 - loss: 0.2218 - val_accuracy: 0.9583 - val_auc_5: 0.0000e+00 - val_loss: 0.169
Epoch 87/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9199 - auc_5: 0.9742 - loss: 0.2107 - val_accuracy: 0.9474 - val_auc_5: 0.0000e+00 - val_loss: 0.179
Epoch 88/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9205 - auc_5: 0.9740 - loss: 0.2145 - val_accuracy: 0.9563 - val_auc_5: 0.0000e+00 - val_loss: 0.180
Epoch 89/100
```

```
from sklearn.metrics import precision_score, recall_score
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
test_loss, test_accuracy, test_auc = model.evaluate(X_test, y_test, verbose=0)
test_precision = precision_score(y_test, y_pred_classes)
test_recall = recall_score(y_test, y_pred_classes)

print(f"- Loss: {test_loss:.4f}")
print(f"- AUC: {test_auc:.4f}")
print(f"- Accuracy: {test_accuracy:.4f}")
```

```
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9154 - auc_5: 0.9721 - loss: 0.2246 - val_accuracy: 0.9578 - val_auc_5: 0.0000e+00 - val_loss: 0.180
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
Epoch 96/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9194 - auc_5: 0.9727 - loss: 0.2180 - val_accuracy: 0.9536 - val_auc_5: 0.0000e+00 - val_loss: 0.183
Epoch 97/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9185 - auc_5: 0.9724 - loss: 0.2176 - val_accuracy: 0.9656 - val_auc_5: 0.0000e+00 - val_loss: 0.176
Epoch 98/100
```

- Loss: 0.3176
- AUC: 0.9525
- Accuracy: 0.8962

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_classes))
```

```
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9192 - auc_5: 0.9756 - loss: 0.2098 - val_accuracy: 0.9615 - val_auc_5: 0.0000e+00 - val_loss: 0.187
              precision    recall  f1-score   support

           0       0.91      0.88      0.90      1065
           1       0.89      0.91      0.90      1065

    accuracy                           0.90      2130
   macro avg       0.90      0.90      0.90      2130
weighted avg       0.90      0.90      0.90      2130
```

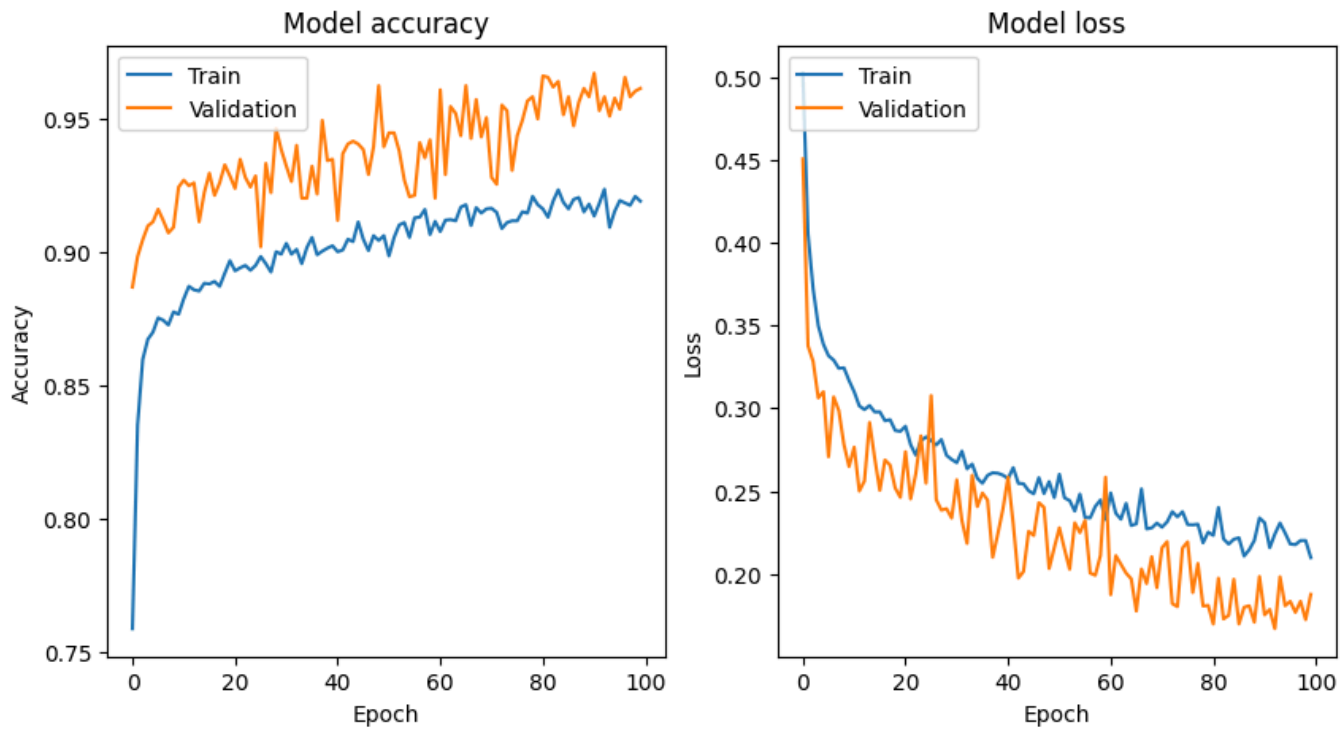
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
```

```
plt.legend(['Train', 'Validation'], loc='upper left')

plt.show()
```



PREDICT

```
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
```

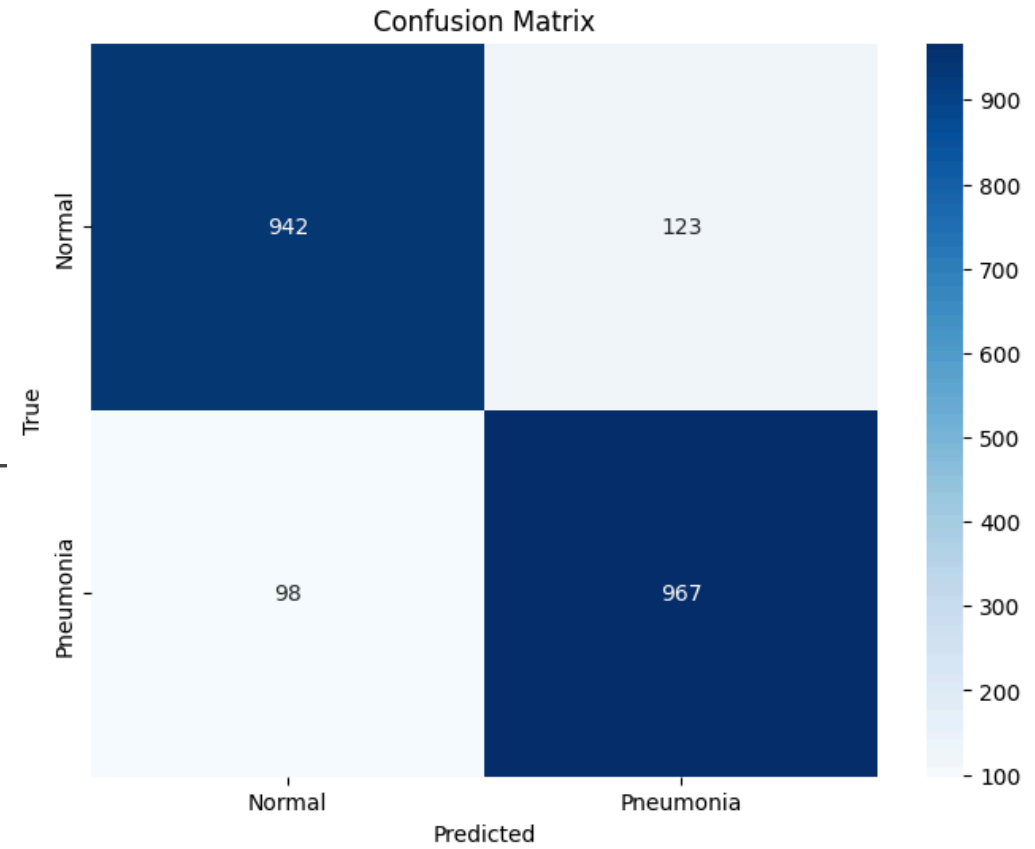
```
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

conf_matrix = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
```

```
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```




```
model.save("Pnem_sim.h5")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We re
```

```
from google.colab import files
files.download("Pnem_sim.h5")
```

```
import pickle

with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)
```

```
files.download("scaler.pkl")
```

CODE END HERE

Tes SVM

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Create and train the SVM model
svm_model = SVC(kernel='linear', random_state=42) # You can choose different kernels
svm_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_svm = svm_model.predict(X_test)

# Evaluate the SVM model
print("\n--- SVM Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_svm):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_svm))

# Confusion Matrix
conf_matrix_svm = confusion_matrix(y_test, y_pred_svm)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_svm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix (SVM)')
plt.show()
```

```
--- SVM Model Evaluation ---
Accuracy: 0.8864

Classification Report:
              precision    recall  f1-score   support

           0       0.87      0.91      0.89      1065
           1       0.90      0.87      0.88      1065

    accuracy                           0.89      2130
   macro avg       0.89      0.89      0.89      2130
weighted avg       0.89      0.89      0.89      2130
```

